



Events



Emit cheatsheet

Version: 4.x

Emit cheatsheet

Server-side

```
io.on("connection", (socket) => {  
  
  // basic emit back to sender  
  socket.emit(/* ... */);  
  
  // to all clients in the current namespace except the sender  
  socket.broadcast.emit(/* ... */);  
  
  // to all clients in room1 except the sender  
  socket.to("room1").emit(/* ... */);  
  
  // to all clients in room1 and/or room2 except the sender  
  socket.to(["room1", "room2"]).emit(/* ... */);  
  
  // to all clients in room1  
  io.in("room1").emit(/* ... */);  
  
  // to all clients in room1 and/or room2 except those in room3  
  io.to(["room1", "room2"]).except("room3").emit(/* ... */);  
  
  // to all clients in namespace "myNamespace"  
  io.of("myNamespace").emit(/* ... */);  
  
  // to all clients in room1 in namespace "myNamespace"  
  io.of("myNamespace").to("room1").emit(/* ... */);  
  
  // to individual socketid (private message)  
  io.to(socketId).emit(/* ... */);  
  
  // to all clients on this node (when using multiple nodes)  
  io.local.emit(/* ... */);  
  
  // to all connected clients  
  io.emit(/* ... */);  
}
```



```
// to all clients, with one acknowledgement per client
io.timeout(5000).emit("hello", "world", (err, responses) => {
  if (err) {
    // some clients did not acknowledge the event in the given delay
  } else {
    console.log(responses); // one response per client
  }
});

// WARNING: `socket.to(socket.id).emit()` will NOT work, as it will send to
everyone in the room
// named `socket.id` but the sender. Please use the classic `socket.emit()`
instead.

// with acknowledgement
socket.emit("question", (answer) => {
  // ...
});

// without compression
socket.compress(false).emit(/* ... */);

// a message that might be dropped if the low-level transport is not writable
socket.volatile.emit(/* ... */);

// with timeout
socket.timeout(5000).emit("my-event", (err) => {
  if (err) {
    // the other side did not acknowledge the event in the given delay
  }
});
});
```

Client-side

```
// basic emit
socket.emit(/* ... */);

// with acknowledgement
socket.emit("question", (answer) => {
```

```
// ...  
});  
  
// without compression  
socket.compress(false).emit(/* ... */);  
  
// a message that might be dropped if the low-level transport is not writable  
socket.volatile.emit(/* ... */);  
  
// with timeout  
socket.timeout(5000).emit("my-event", (err) => {  
  if (err) {  
    // the other side did not acknowledge the event in the given delay  
  }  
});
```

Reserved events

On each side, the following events are reserved and should not be used as event names by your application:

- `connect`
- `connect_error`
- `disconnect`
- `disconnecting`
- `newListener`
- `removeListener`

```
// BAD, will throw an error  
socket.emit("disconnecting");
```

 [Edit this page](#)

Last updated on **Jun 20, 2023**